Architecture Decision Record

ADR-002: State-Management und Datenaktualisierung in der Chat-App

Gruppe 5 · Mobile UX, Hochschule Esslingen · 12. Oktober 2026

Status: Vorgeschlagen

Baut auf: ADR-001 (Wahl von React 19 mit Vite & TypeScript)

1 Titel und Nummer

ADR-002: State-Management und Datenaktualisierung in der Chat-App

2 Status

Vorgeschlagen

3 Kontext

Mit ADR-001 wurde React 19 mit Vite und TypeScript als Frontend-Framework festgelegt. Offen ist, **wie der Anwendungszustand verwaltet** wird und **wie neue Daten vom Chat-Server** (`https://www2.hs-esslingen.de/~chkohl/chat/`) in die Oberfläche gelangen.

Zu verwaltender Zustand

- **Authentifizierung:** Token nach der Registrierung/Anmeldung. Es muss über Seiten-Reloads erhalten bleiben und an jeden API-Aufruf angehängt werden.
- **Chat-Daten:** Nachrichten des aktiven Raums, Raumliste, Mitglieder, Suchergebnisse.
- **UI-Zustand:** Aktiver Tab (*API-Test* / *Live-Chat*), geöffnete Modals, Formulareingaben, Lade- und Fehlerzustände.
- **API-Testbench:** Fortschritt des 4-stufigen Ablaufs, Zeitstempel für die 1s-Delay-Validierung, empfangene HTTP-Fehlercodes (451–469).

Randbedingungen

- Der Chat-Server bietet eine reine HTTP-Schnittstelle (GET mit Query-Parametern / POST mit JSON-Body). Neue Nachrichten müssen daher **per Polling** abgefragt werden.
- Der Zustand wird nur von wenigen Komponenten gemeinsam genutzt (Header, Chat-Seite, Modals).
- Der Code soll übersichtlich bleiben und ohne lange Einarbeitung nachvollziehbar sein (vgl. ADR-001).

4 Optionen und Bewertung

Es wurden drei Optionen für das State-Management betrachtet:

1. **Option A: React-Bordmittel** (`useState` , `useReducer` , `useRef` , Context API), gekapselt in Custom Hooks (`useAuth` , `useChat`)
2. **Option B: Zustand** (schlanke externe State-Bibliothek)
3. **Option C: Redux Toolkit mit RTK Query**

Gewichtete Entscheidungsmatrix

Bewertungsskala: 1 (schlecht) bis 5 (sehr gut). Der Gesamtwert ergibt sich als Summe aus *Gewichtung* $\times$ *Punktzahl*. Die gewählte Option ist grün hinterlegt.

Kriterium	Gewichtung	A: React	B: Zustand	C: Redux
Geringer Boilerplate & Übersichtlichkeit	25 %	5	4	2
Eignung für Polling & Token-Handling	25 %	4	4	5
TypeScript-Integration	20 %	5	5	4
Keine zusätzlichen Abhängigkeiten	15 %	5	3	2
Skalierbarkeit & Debugging-Tools	15 %	3	4	5
Gesamtbewertung	**100 %**	**4,45**	**4,05**	**3,60**

Tabelle 1: Gewichtete Entscheidungsmatrix (Punktzahlen 1–5)

Rechenweg:

$$\begin{aligned}
\text{A:}\quad & 0{,}25 \cdot 5 + 0{,}25 \cdot 4 + 0{,}20 \cdot 5 + 0{,}15 \cdot 5 + 0{,}15 \cdot 3 = \mathbf{4{,}45} \\
\text{B:}\quad & 0{,}25 \cdot 4 + 0{,}25 \cdot 4 + 0{,}20 \cdot 5 + 0{,}15 \cdot 3 + 0{,}15 \cdot 4 = \mathbf{4{,}05} \\
\text{C:}\quad & 0{,}25 \cdot 2 + 0{,}25 \cdot 5 + 0{,}20 \cdot 4 + 0{,}15 \cdot 2 + 0{,}15 \cdot 5 = \mathbf{3{,}60}
\end{aligned}$$

Detaillierte Bewertung der Alternativen

- **Redux Toolkit mit RTK Query (Option C):**
  - *Vorteile:* Eingebautes Polling (`pollingInterval`), Caching, sehr gute DevTools, bewährt in großen Projekten.
  - *Nachteile:* Hoher Einarbeitungs- und Konfigurationsaufwand (Store, Slices, API-Definitionen) – für den Umfang der Übung überdimensioniert.
- **Zustand (Option B):**
  - *Vorteile:* Sehr wenig Code, globaler Store ohne Provider, gute TypeScript-Unterstützung, `persist` -Middleware für das Token.
  - *Nachteile:* Zusätzliche Abhängigkeit. Bei wenigen gemeinsam genutzten Zuständen ist der Vorteil gegenüber React-Bordmitteln gering.
- **React-Bordmittel in Custom Hooks (Option A):**

  - *Vorteile:* Keine zusätzliche Bibliothek. Die Logik ist sauber in `useAuth` und `useChat` gekapselt. Polling über `useEffect` + `setInterval`, mit `useRef` für den letzten Nachrichtenstand, ist gut kontrollierbar.
  - *Nachteile:* Kein eingebautes Caching. Bei stark wachsendem Zustand kann Context unnötige Re-Renders verursachen.

5 Entscheidung

Entscheidung

Wir entscheiden uns für **Option A: React-Bordmittel, gekapselt in Custom Hooks**.

Umsetzung im Detail:

- `useAuth` **(via** `AuthContext`**):** Hält das Token, speichert es in `localStorage` und stellt `login`, `register` und `logout` bereit. Der Context wird in der App-Wurzel bereitgestellt, da Header, Chat und API-Test das Token benötigen.
- `useChat`**:** Verwaltet Nachrichten, Räume und Mitglieder des aktiven Raums. Neue Nachrichten werden per **Polling** abgefragt (`setInterval` in `useEffect`, Aufräumen im Cleanup). Über `useRef` wird die ID bzw. der Zeitstempel der letzten Nachricht gemerkt, sodass nur neue Nachrichten angehängt werden.
- **Lokaler UI-Zustand** (Modals, Formulareingaben, aktiver Tab) bleibt per `useState` in der jeweiligen Komponente.
- `src/api/` bleibt zustandslos. Die Hooks rufen die API-Funktionen auf und übergeben das Token explizit.

6 Konsequenzen

Positive Folgen

- **Keine zusätzlichen Abhängigkeiten:** Das Projekt bleibt schlank, der Build schnell.
- **Klare Verantwortlichkeiten:** Komponenten zeigen nur an, Logik und Zustand liegen in den Hooks – passend zur Schichtenarchitektur aus ADR-001.
- **Nachvollziehbares Polling:** Start, Stopp und Intervall sind explizit im Code sichtbar. Beim Raumwechsel oder Logout wird das Intervall sauber beendet.
- **Persistente Anmeldung:** Durch `localStorage` bleibt das Token über Seiten-Reloads erhalten.

Negative Folgen & Risiken

- **Polling-Last:** Regelmäßige Anfragen erzeugen Netzwerkverkehr, auch wenn keine neuen Nachrichten vorliegen. Das Intervall muss passend gewählt werden (z. B. 2–3 s).
- **Fehleranfälligkeit bei `useEffect`:** Fehlendes Cleanup führt zu doppelten Intervallen (insbesondere im React-StrictMode) oder zu veralteten Werten in Closures.
- **Token in `localStorage`:** Anfällig bei XSS. Für die Übung akzeptabel, im Produktivbetrieb wären HttpOnly-Cookies vorzuziehen.
- **Migrationspfad:** Wächst der Zustand deutlich, ist ein Wechsel zu Zustand (Option B) mit geringem Aufwand möglich, da die Logik bereits in Hooks gekapselt ist.